

ML Pipeline

Neural operator ML pipeline

<https://github.com/tglanz/climatology/tree/main/ml>

Data synthesis

Generate training data from physics-based simulations

Simulations

Run numerical models to generate ground truth datasets

- ISCA: Earth-scale atmospheric circulation
 - <https://github.com/execlim/isca>
- Dynamics: Barotropic vorticity equation with forcing (stirring)
 - T85, 128 x 256
 - 15 segments of 7 days (105 snapshots)
- Runs produce snapshots on NetCDF files at
`<experiment>/simulations/<index>/runs*`
- Output variables: vorticity ω , stirring F field

Preprocessing, training pairs

Create (x, y) pairs for training based on the simulation snapshots.

Each simulation (trajectory) is chunked to fixed K -length windows (tumbling with stride 1).

For window i that starts at snapshot j

- $x_i: (\omega_j, F_j, \omega_{j+1}, F_{j+1}, \dots, \omega_{j+K-1}, F_{j+K-1})$
- $y_i: \omega_{j+K}$

Preprocessing, training split

- Split the training pairs into
 - `train.h5`: affects weight updates
 - `val.h5`: used for loss evaluation during training
 - `test.h5`: used for evaluating the model on unseen samples after training

Split files and a `split.json` listing the simulation sources are saved at `<experiment>/data`.

Model

Neural Operator

Model, FNO

Fourier Neural Operator

- Lift input to higher channels with (channel-mixing) MLP (P layer)
- Apply spectral convolution blocks: FFT, multiply by learned weights, IFFT
- Project output back with MLP (Q layer)

Fourier Neural Operator for Parametric Partial Differential Equations

Model, SFNO

Spherical Fourier Neural Operator: FNO adapted for spherical geometry

- Operates on spherical coordinates using spherical harmonics basis
- Maintains stability and symmetries on the sphere
- Designed for global climate and weather data
 - [FourCastNet3](#)

[Spherical Fourier Neural Operators: Learning Stable Dynamics on the Sphere](#)

Training

Training, Normalization

Standardize input and target data using z-score normalization fitted on training set

- UnitGaussianNormalizer: compute mean and std from training data
- Fit separately per channel (for both x and y)
- Save normalization statistics for inference at
`<experiment>/training/normalization.pt`

Training, Optimization

Iteratively update model weights using AdamW optimizer

- Adam with weight decay (AdamW)
- CosineAnnealingLR scheduler to decay learning rate
- Loss: latitude-weighted relative L2
- Early stopping (target loss, patience)

Training, Regularization

Prevent overfitting via weight penalties and stochastic depth

- Weight decay (by AdamW)
- Dropout in the channel-mixing MLP layers

Evaluation

Assess model performance on held-out test data

Evaluation, Autoregressive rollout

Predict vorticity by iteratively feeding predictions back into model as input

- Input: K historical vorticity snapshots + $K + (T - 1)$ stirring snapshots
- For each of the T prediction steps:
 - Normalize input and predict next vorticity
 - Slide window: drop oldest vorticity, append new prediction
- Output: final vorticity prediction at time T

Current Status

Model exhibits overfitting and doesn't generalize. Hypothesis - Under the current architecture (deterministic Neural Operator), the model averages the stochastic forcing.

“ For operator learning to be mathematically meaningful, the target mapping must be well-posed in the following sense: each admissible input function $f \in F_{in}$ must correspond to a unique output $g \in F_{out}$. If the mapping is not unique, i.e. multiple valid outputs exist for the same input, a deterministic neural operator cannot represent it consistently. In such cases, the model tends to average across possible outputs, which often leads to blurred or unstable predictions and poor generalization. When non-uniqueness is inherent to the problem, alternative probabilistic formulations or ensemble-based neural operators are required...”

[Duruiseaux, Kossaifi, Anandkumar: Fourier Neural Operators Explained: A Practical Perspective](#)

Next Steps

- BVE with deterministic forcing
- SFNO architecture
 - <https://arxiv.org/abs/2306.03838>

Possible thesis Directions?

1. Stochastic forcing: incorporate noise in simulation and learning
2. Qualitative feature prediction: predict large-scale structures (jets, Rossby waves, vortices, cyclones, fronts) instead of pointwise fields
3. Generalization to forcings: extend to varying forcing patterns
4. Architecture enhancement: design operators better suited to this domain
5. Model generalization: learn across different physical parameters (Reynolds number, viscosity, forcing amplitude)
6. Inverse problems: infer forcing, parameters, or initial conditions from observations
7. Super resolution: predict higher resolution from coarse input
8. Computational enhancement: distributed training / accelerated training